Urban Mobility Data Explorer – Technical Report

NYC Yellow Taxi Analytics, January 2019 | Team: Manzi Elvis (Frontend), Daniel Kenny Kamanzi (Database), Olivier Dusabamahoro (Backend) | [Course / date]

1. Problem Framing and Dataset Analysis

Dataset and context. We use the NYC Taxi & Limousine Commission (TLC) Yellow Taxi trip records for January 2019 (*yellow_tripdata_2019-01.csv*, about 690 MB, the fact table) joined to *taxi_zone_lookup.csv* (265 zones, the dimension table). Each trip has pickup/dropoff times and zones, distance, passenger count, itemised charges, tip and payment type. The goal is an explorer that shows how people move through the city: when, where, and at what cost.

Data challenges. Of 7,667,792 raw rows, 197,928 (2.6%) were rejected, leaving 7,469,864 clean trips. Problems found:

- **Invalid values:** zero passengers (115,608), zero or negative distance (47,350), negative fares (7,122).
- **Anomalies:** dropoff at or before pickup (6,291), pickups outside the reporting month (537), speeds above 100 mph (5,211), and a stuck meter that ran 1–31 January.
- **Outliers:** distances over 100 mi (29), fares over $500 (9), trips over 24 h (5).
- **Missing fields:** congestion_surcharge is blank for the whole file and airport_fee does not yet exist in 2019 files; zone placeholders 264 and 265 have no real names.

Cleaning assumptions. Each row is labelled with the *first* rule it breaks and written to a dead-letter file with its reason, so nothing is silently lost. Missing passenger_count and RatecodeID default to the mode (1); missing surcharges default to 0; zone 264 becomes "Unknown" and 265 "Outside NYC". Speed is checked after feature engineering. Whitespace and duplicate or non-numeric zone IDs are cleaned in the lookup table.

Unexpected observation. 15,766 trips had a total_amount that did not match the sum of their itemised charges by more than $1. Almost all came from vendor 2 and were off by exactly $1.95, $2.50 or $4.95, which points to a surcharge billed before it had its own column rather than random noise. We excluded these rows because their totals cannot be reconciled, but we gave them their own rejection reason and documented the pattern, so the exclusion is a visible, auditable choice and the Data Quality tab can show it.

2. System Architecture and Design Decisions

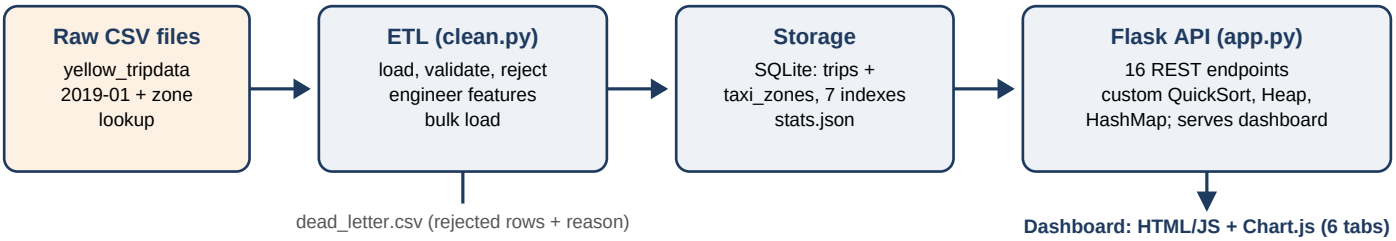

Figure 1. CSV files pass through the ETL pipeline into SQLite and a precomputed stats file; the Flask API serves JSON and the dashboard.

- **Stack:** SQLite needs no database server, so anyone can clone and run the project. Flask is small enough for a read-heavy JSON API and also serves the dashboard, so one command starts everything. Vanilla JS with Chart.js needs no build step; the frontend is split into ES modules (one per tab).
- **Schema:** two tables, *taxi_zones* (location_id, borough, zone, service_zone) and *trips* (raw TLC fields plus derived duration, speed_mph, fare_per_mile, tip_percentage, hour, day_name, is_airport). Seven indexes cover pickup_date, pu_borough, pu_zone, hour, payment_type, is_airport and fare_amount.
- **Trade-offs:** (1) Borough and zone names are denormalised onto each trip, which duplicates text over 7.5M rows but avoids a join in every filter and group-by. (2) Dashboard statistics are precomputed into stats.json, which is fast but can go stale until the ETL is rerun. (3) SQLite is easy to deploy but limits concurrent writes and scale. (4) Rejecting fare mismatches loses real trips but keeps revenue figures reconcilable. (5) The full ETL needs about 4 GB of RAM because it loads the whole file.

3. Algorithmic Logic and Data Structures

We implemented three algorithms by hand in *backend/etl/algorithms.py*, with no built-in sort, heapq or Counter. They are run live against the database by */api/dsa-benchmark*, which times them against the Python built-ins and checks the outputs match.

Algorithm	Used for	Time / space
QuickSort (Lomuto, descending)	Sorting trips by fare	O(n log n) average, O(n^2) worst; O(log n) stack, since it recurses only into the smaller half
Min-Heap Top-K	Top 10 busiest pickup zones	O(n log k) time, O(k) space
HashMap (separate chaining, polynomial rolling hash)	Trips per borough	O(n) average, O(k) space

Featured algorithm: Min-Heap Top-K. *Problem:* the Geography tab needs the 10 busiest pickup zones out of millions of trips. Sorting all zones costs O(Z log Z) and extra memory. *Approach:* keep a min-heap of size K whose root is the smallest of the current top K. Each zone count is compared with the root and replaces it only if larger, so the heap never exceeds K items. Popping the heap at the end gives the result in ascending order, which we reverse.

```
# Pseudo-code
counts = HashMap();  for each trip: counts[trip.pu_zone] += 1          # O(n)
heap = empty MinHeap
for (zone, c) in counts:
    if size(heap) < K:        push(heap, (c, zone))                     # O(log K)
    else if c > root(heap).c: replace_root(heap, (c, zone)); sift_down  # O(log K)
result = [];  while heap not empty: result.append(pop(heap))
return reverse(result)                                                  # descending
```

Complexity. Counting is O(n); the heap phase touches each of the k distinct zones with O(log K) work, giving **O(n log k)** time as measured in our benchmark, and **O(k)** space for the heap. Because K is fixed at 10, the heap adds very little overhead.

4. Insights and Interpretation

Insert screenshot 1

Time Patterns tab:
trips by hour of day

Insert screenshot 2

Geography tab:
Top-10 pickup zones

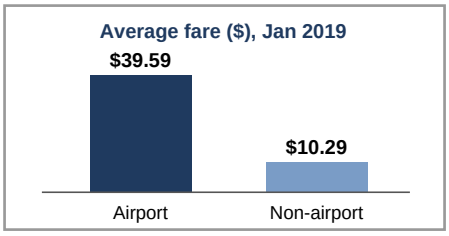

Figure 2. Left and centre: replace with dashboard screenshots. Right: average fare, airport vs other trips (from /api/airport).

Insight 1 – Demand peaks in the early evening. *Derived via* a GROUP BY on the indexed *hour* column (*/api/by-hour*) shown as a bar chart. The busiest hour is 18:00 with 501,112 trips. *Meaning:* the peak lines up with the evening commute, so taxis act as a last-mile or overflow option when transit is most crowded, which matters for scheduling and driver supply.

Insight 2 – Demand is concentrated in Manhattan's core. *Derived via* our custom Min-Heap Top-K (*/api/top-zones*). The leaders are Upper East Side South, Upper East Side North and Midtown Center. *Meaning:* a few dense residential and business districts generate a large share of pickups, so curb management and transit investment there would reach the most riders.

Insight 3 – Airport trips are far more valuable but the city is slow. *Derived via* an airport flag (*is_airport*) and an average-fare query (*/api/airport*): airport trips average $39.59 versus $10.29 for all others, nearly 4 times as much. Overall, trips average 2.82 mi in 16.6 min at only 11.7 mph. *Meaning:* airports are high-value anchors for the taxi system, while the low average speed shows how congestion limits what short city trips can deliver.

5. Reflection and Future Work

Technical challenges. Processing 7.7M rows needed about 4 GB of RAM and a few minutes, so we added a *--nrows* option for fast testing. Choosing rejection rules without ground truth was difficult, especially for the vendor-2 fare mismatches. We also ported an earlier version of the project to the 2019 file format, which required reworking the ETL, API validation and tests (we wrote tests for the cleaning, algorithms and API). **Team challenges.** [Add 1–2 sentences: e.g. keeping frontend, backend and database in sync, how you split work and resolved conflicts].

Next steps for a real product.

- Process data in chunks or use Parquet/PostgreSQL with caching, so the pipeline needs less memory and handles concurrent users.
- Add more months and years, and map views using zone geometry (heatmaps, origin–destination flows).
- Replace fixed thresholds with statistical anomaly detection, and add authentication, CI/CD and monitoring.

AI usage disclosure. AI assistance (Claude Code) was used to port the earlier version of this project: adapting the ETL to the 2019 file format, the API validation, the tests and the README.